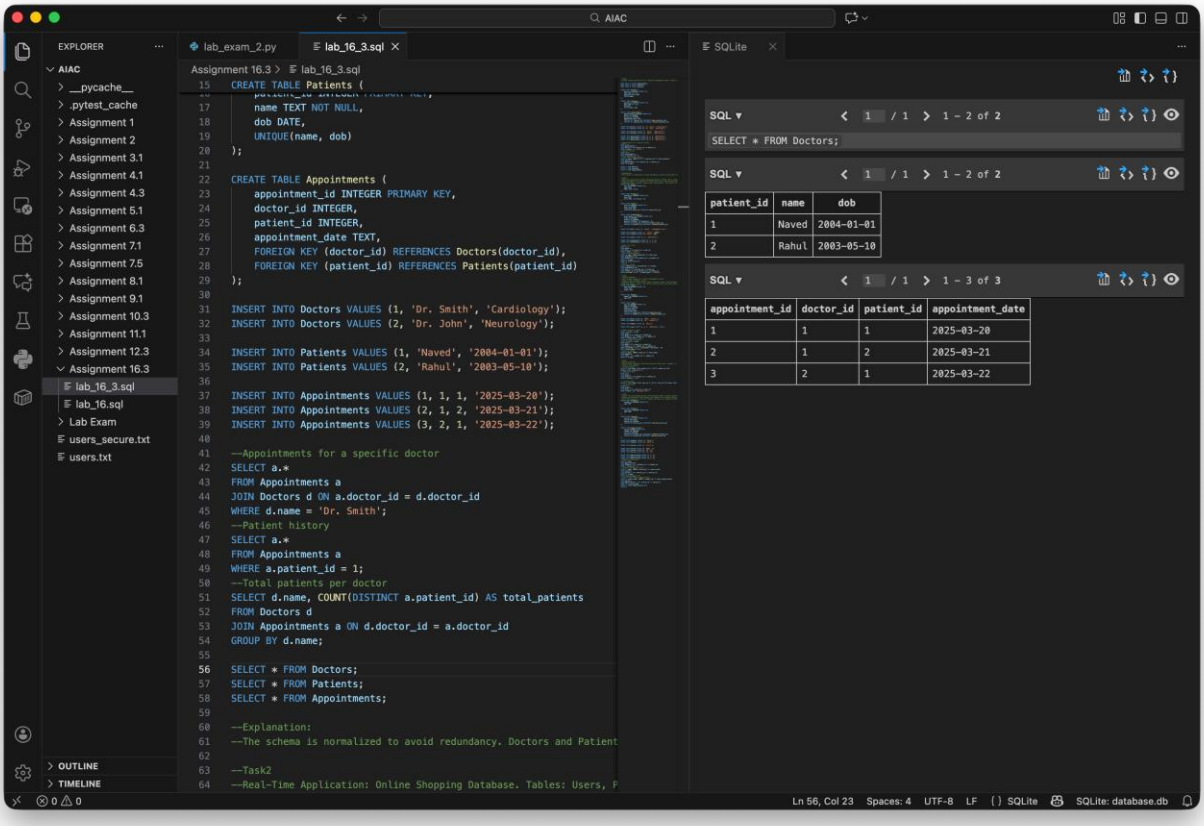
School of Computer Science and Artificial Intelligence

Lab Assignment - 12.3

Course Name : AI Assistant Coding
Name of the Student : MOHAMMAD ADIL
Registration No : 2303A54042
Batch No : 48

Task 1:Hospital Management Database

Task:
Create schema and queries for a Hospital Management System.
Instructions:
• Tables: Doctors, Patients, Appointments.
• Use AI to define constraints (unique IDs, valid dates).
• Generate queries:
• List all appointments for a specific doctor.
• Retrieve patient history by patient ID.
• Count total patients treated by each doctor. Expected Output:
• Normalized schema and SQL queries with joins.



Explanation:

The schema is normalized to avoid redundancy. Doctors and Patients have unique constraints to prevent duplicate entries. Appointments link doctors and patients with foreign keys. Queries utilize joins to retrieve relevant data efficiently, ensuring accurate results while maintaining data integrity.

Task 2:Real-Time Application: Online Shopping Database

Scenario:

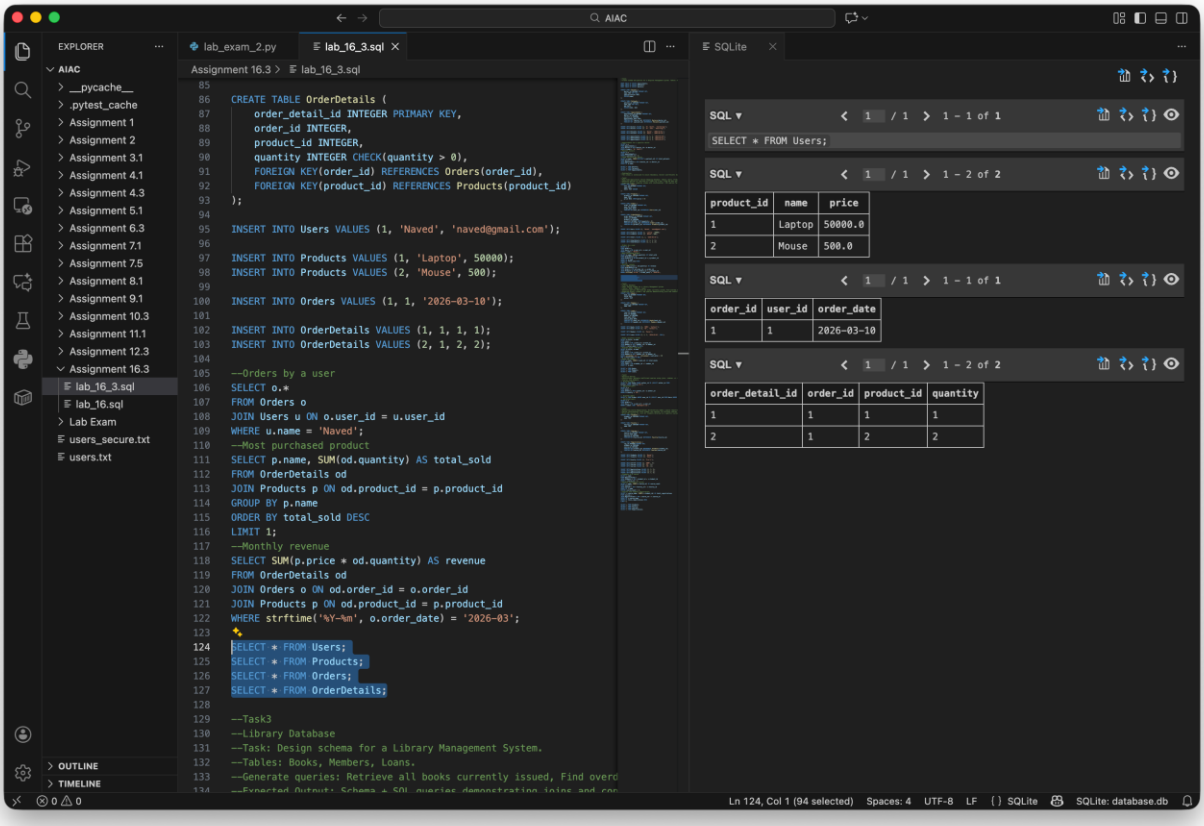
Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
- Retrieve all orders by a user.
- Find the most purchased product.
- Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.



Explanation: The schema is designed to maintain data integrity and avoid redundancy. Users and Products have unique constraints to prevent duplicates. Orders and OrderDetails link users and products with foreign keys. Queries are optimized using joins to efficiently retrieve relevant data, ensuring accurate analytics while maintaining performance. Normalization ensures that data is stored in a structured manner, reducing redundancy and improving data integrity.

Task 3:Library Database

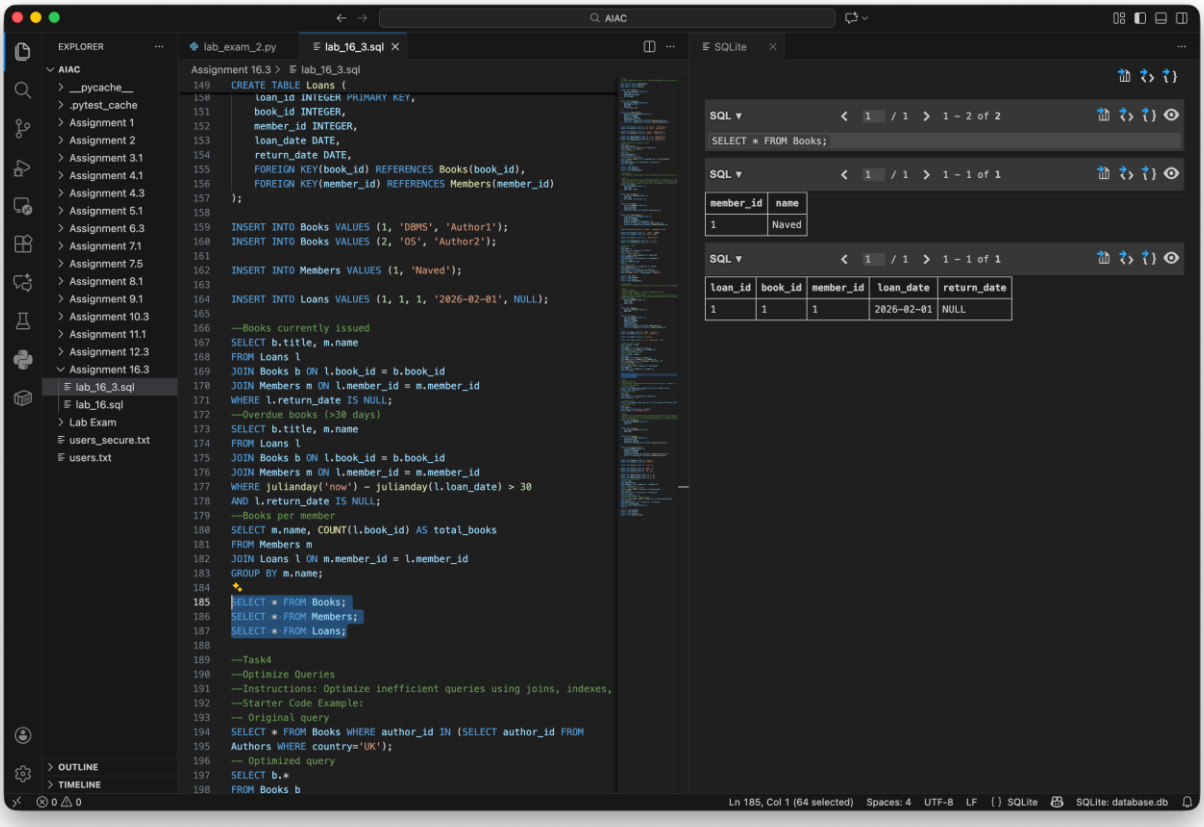
Task:

Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:

• Retrieve all books currently issued.
• Find overdue books (loan date > 30 days).
• Count number of books loaned by each member. Expected Output:
• Schema + SQL queries demonstrating joins and conditions.



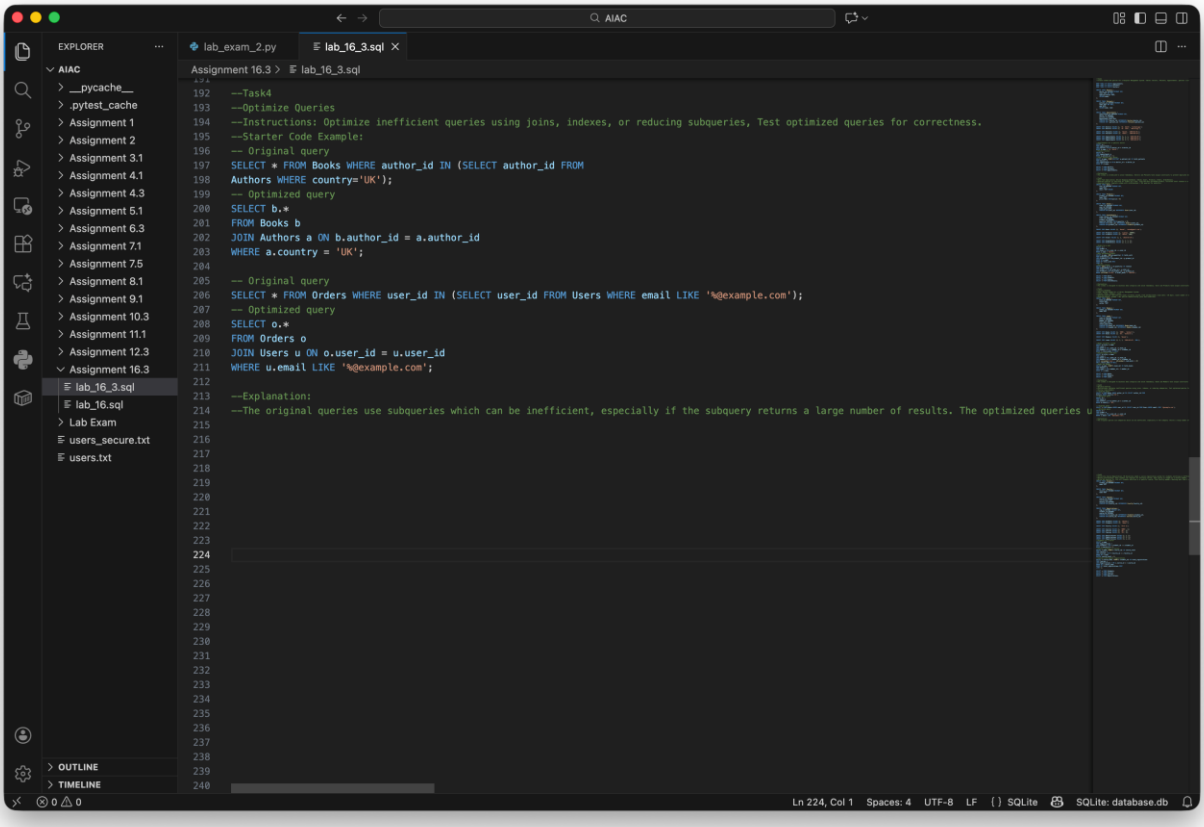
Explanation: The schema is designed to maintain data integrity and avoid redundancy. Books and Members have unique constraints to prevent duplicates. Loans link books and members with foreign keys, allowing for efficient tracking of issued books. Queries utilize joins to retrieve relevant data based on conditions, ensuring accurate results while maintaining performance. The use of conditions in queries allows for filtering data effectively, such as identifying overdue books and counting the number of books loaned by each member.

Task 4: Optimize Queries

Instructions:
• Use AI tools to analyze query efficiency.
• Optimize inefficient queries using joins, indexes, or reducing subqueries.
• Test optimized queries for correctness.
Starter Code Example:

```
-- Original query
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM
Authors WHERE country='UK');
-- Optimized query
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id =
a.author_id WHERE a.country = 'UK';
```

Expected Output:
• Both queries return the same result: all books written by UK authors.
• Optimized query performs faster for larger datasets.



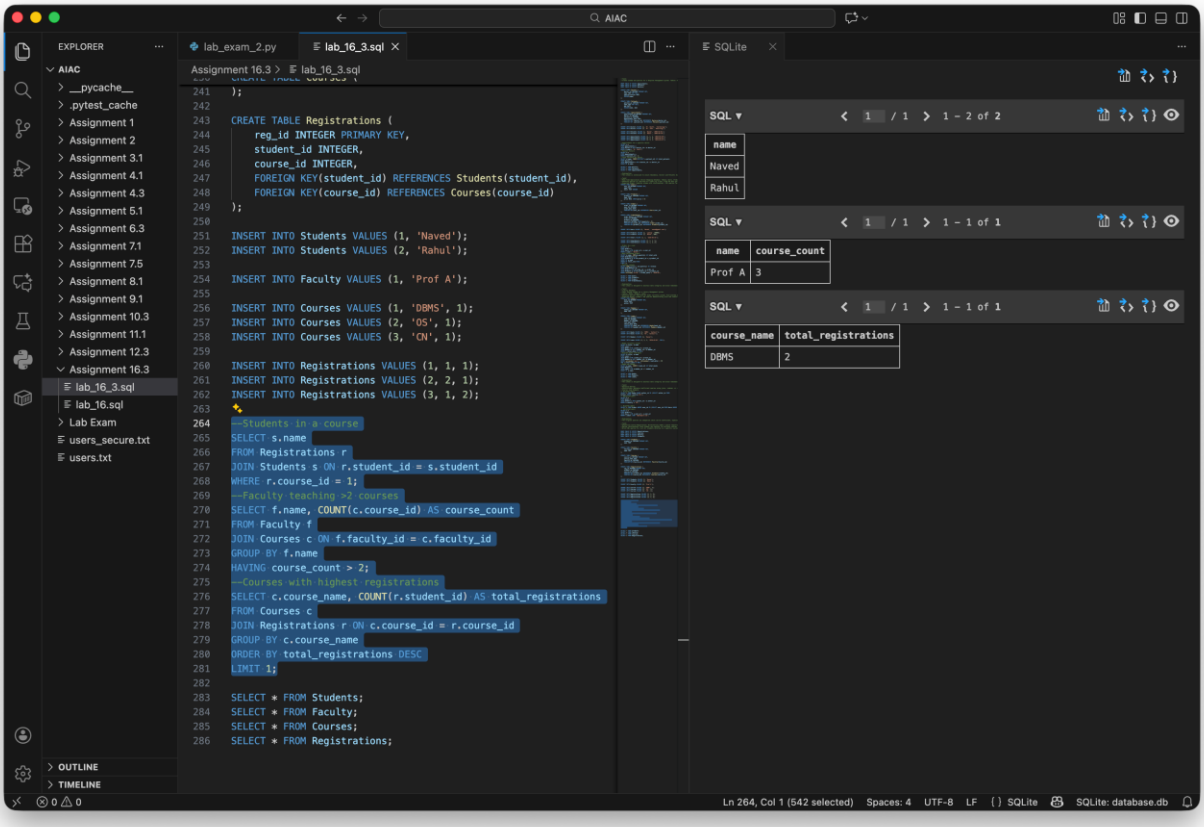
Explanation: The original queries use subqueries which can be inefficient, especially if the subquery returns a large number of results. The optimized queries utilize joins, which are generally more efficient for retrieving related data. By joining the relevant tables directly, we can filter results more efficiently and reduce the overhead associated with subqueries. Additionally, ensuring that appropriate indexes are in place on the columns used in joins and where conditions can further enhance query performance. Testing the optimized queries for correctness ensures that they return the expected results while improving efficiency.

Task 5:Task 5: University Course Registration

Scenario:

SR University needs a course registration system for students enrolling in different courses under faculty supervision.

• Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
• Define relationships:
• Each student can register for multiple courses.
• Each course is taught by a faculty member.
• Write SQL queries to:
• List all students enrolled in a specific course.
• Find faculty members teaching more than 2 courses.
• Retrieve courses with the highest number of registrations.



Explanation: The schema is designed to maintain data integrity and avoid redundancy. Students and Faculty have unique constraints to prevent duplicates. Courses link to Faculty with a foreign key, and Registrations link Students and Courses with foreign keys, allowing for efficient tracking of course enrollments. Queries utilize joins to retrieve relevant data based on conditions, ensuring accurate results while maintaining performance. The use of GROUP BY and HAVING clauses allows for aggregation and filtering of results, such as identifying faculty members teaching more than 2 courses and retrieving courses with the highest number of registrations.